

PEARL: Adaptive Prefill–Decode Execution with Elasticity for Agentic Reinforcement Learning

Jiaan Zhu*
zja_pb17151780@mail.ustc.edu.cn
USTC
China

Zewen Jin
zevin@ustc.edu.cn
USTC
China

Jiamang Wang
jiamang.wang@alibaba-inc.com
Alibaba Group
China

Wei Gao*
csgaowei@ust.hk
HKUST
China

Ju Huang
huangju.hj@alibaba-inc.com
Alibaba Group
China

Lin Qu
xide.ql@taobao.com
Alibaba Group
China

Youhui Bai[✉]
youhuibai@ustc.edu.cn
USTC
China

Siran Yang
siran.yyr@alibaba-inc.com
Alibaba Group
China

Cheng Li
chengli7@ustc.edu.cn
USTC
China

Abstract

Multi-turn rollout dominates the cost of agentic reinforcement learning (RL). Asynchronous execution and elastic GPU resources can accelerate this stage, but adding rollout replicas yields diminishing returns while training GPUs remain idle between updates. We observe that effective resource use also depends on the prefill–decode (PD) configuration. Both the choice between colocation and disaggregation and the optimal PD ratio vary with the workload, making resource scaling and PD configuration interdependent. Exploiting this opportunity requires selecting effective configurations and realizing their benefits within transient resource-availability windows despite reconfiguration costs.

We present PEARL, an asynchronous agentic RL system that coordinates external resource elasticity, temporary reuse of idle training GPUs, and adaptive PD execution. PEARL maintains a unified GPU–worker–role state and uses runtime profiles to predict rollout batch completion time, accounting for environment-induced reductions in decode concurrency. It selects the PD mode and ratio under the current GPU budget and translates each decision into an incremental transition plan that minimizes worker and role changes. Cost-aware switching and borrowing policies suppress transitions with insufficient expected benefit while ensuring timely return of training GPUs. Our evaluation shows that PEARL achieves 2.17–2.79× the throughput of fixed-resource ROLL across different LLMs. Compared with RLBoost+, throughput improves by up to approximately 26.9% for Qwen3-8B and 36.3% for Qwen3-30B-A3B.

1 Introduction

Agentic reinforcement learning (RL) equips large language models (LLMs) with capabilities such as coding, web navigation, and tool use through interaction with an environment [3, 30]. Its workflow comprises two stages, rollout, which generates interaction trajectories, and training, which updates the model using these trajectories. Rollout proceeds over multiple turns, repeatedly alternating between response generation and environment feedback [31, 36]. Repeated inference over growing contexts and sequential environment interactions make rollout the dominant cost in this workflow [7, 31, 38].

To alleviate this bottleneck, recent RL systems increasingly adopt asynchronous execution, overlapping rollout and training on separate GPU pools [6, 35]. This approach is also used in industrial model training [10, 32, 39]. However, overlap alone does not eliminate the throughput imbalance between the two stages. Systems therefore exploit elastic resources, including preemptible cloud instances and spare capacity in shared clusters, to expand the rollout pool when additional GPUs become available [5, 7, 38, 42].

Existing systems nevertheless struggle to translate this additional capacity into efficient rollout execution (see §3.4). Our characterization of RLBoost+, which extends RLBoost [38] to support asynchronous agentic RL, reveals sublinear scaling. Increasing rollout resources from 16 to 24 GPUs adds 50% more GPUs but reduces rollout time by only 10.3% (Figure 2a). This is because rollout typically scales to additional GPUs through data parallelism [4], which partitions a fixed trajectory batch across more workers while replicating model weights. Each worker therefore processes fewer trajectories while incurring similar weight-access costs, reducing GPU efficiency. Rollout still takes 4.5–5.7× as long as training, leaving training GPUs idle after each update while they await the next trajectory batch. These idle windows expose

*Both authors contributed equally to this research.

another source of rollout capacity. We refer to external resource changes as inter-scale elasticity and temporary reuse of idle training GPUs for asynchronous RL as intra-scale elasticity.

Agentic rollout extends LLM inference with environment interactions, and each inference call consists of two phases, prefill and decode. Our measurements further show that effective resource use depends on how these two phases are executed. First, the preferred PD execution mode depends on the workload. Under PD colocation, recurrent prefills in multi-turn rollout interfere with decoding even with chunked prefill [1] and prefix caching [9] enabled (Figure 2b). PD disaggregation isolates the two phases on separate GPU sets, but introduces KVCache transfers and partitions the available capacity. Second, when disaggregation is preferable, the optimal PD ratio also varies with the workload, requiring adaptive allocation of prefill and decode workers. Because resource arrivals and reclamations change the workload per worker and feasible allocations, both the PD mode and ratio must adapt to the current workload and GPU budget.

Coordinating resource elasticity with PD execution presents two challenges. First, the system must select a configuration that shortens rollout batch completion time under changing workloads and resources. GPU count alone is insufficient to predict this outcome because performance also depends on PD interference, effective decode concurrency, KVCache transfers, and pauses for environment interactions. Second, the system must realize the selected configuration within transient resource-availability windows. Reconfiguration involves worker initialization, weight synchronization, and handling unfinished trajectories, while borrowed training GPUs must be returned before the next update. These transition costs can outweigh the expected acceleration. Configuration selection must therefore account for both execution performance and the cost of switching, while respecting mandatory resource returns.

To this end, we present PEARL, an asynchronous agentic RL system that jointly manages resource elasticity and adaptive PD execution. PEARL represents both external scaling and training-GPU reuse through a unified GPU-worker-role state. Its PD Decision Engine uses runtime profiles to predict rollout batch completion time, accounting for environment-induced reductions in decode concurrency, and selects the execution mode and PD ratio under the current GPU budget. Switching thresholds suppress changes with insufficient expected benefit. A unified orchestration layer converts each target configuration into an incremental transition plan that minimizes worker and role changes while preserving unaffected workers. PEARL initializes added workers in the background, synchronizes their weights before admission, and retains partial trajectories when workers leave. For training-GPU reuse, it retains worker processes across handoffs and activates borrowing only when the predicted benefit exceeds the activation and return costs.

We implement PEARL atop ROLL [35] and evaluate it with Qwen3-8B and Qwen3-30B-A3B [40] on SWE-bench [16]. Experiments replay real preemptible-GPU availability traces on up to 32 GPUs, with 16 reserved for training, four reserved for rollout, and up to 12 additional GPUs available to rollout. Training uses FSDP2, and rollout workers use tensor parallelism of degree two. Across the evaluated traces, PEARL achieves 2.17–2.37 $\times$ the throughput of fixed-resource ROLL for Qwen3-8B and 2.43–2.79 $\times$ for Qwen3-30B-A3B. Compared with RLBoost+ under the same resource-availability traces, PEARL improves throughput by up to approximately 26.9% and 36.3%, respectively.

2 Background

2.1 Agentic RL and Asynchronous Training

Agentic reinforcement learning (RL) trains large language models (LLMs) to solve tasks through repeated interaction with an environment [3, 30, 36]. By optimizing policies with rewards from these interactions, it develops capabilities such as planning, tool use, and adapting actions to feedback, with applications in software engineering [12, 16, 37, 41] and computer use [19–21]. As shown in Figure 1a, a typical agentic RL workflow comprises two stages, rollout and training. Rollout executes the policy to generate samples, while training consumes these samples to update model weights. The updated weights are then transferred back to rollout for the next step of workflow.

Figure 1b zooms in on the rollout stage, which combines LLM inference with multi-turn environment interaction. A task from the training dataset supplies the initial prompt p_0 . Prefill processes it, and autoregressive decoding produces response r_0 . The environment executes the action and returns observation o_0 , yielding $p_1 = p_0 \oplus r_0 \oplus o_0$ for the next turn. Interaction continues until task completion, forming a trajectory that training uses to update model weights.

Multi-turn interaction makes rollout particularly expensive. Responses and observations accumulate across turns, expanding the context processed by subsequent inference calls, while autoregressive generation and environment execution introduce sequential dependencies within each trajectory. Together, these costs make rollout the dominant stage in our evaluated workload. For example, in our Qwen3-8B experiment on SWE-bench (Figure 2a), rollout still accounts for 81.8% of the end-to-end agentic RL execution time, even when allocated four times as many GPUs as training.

To improve workflow efficiency, frameworks like AReaL [6] and ROLL [35] support asynchronous RL, an approach increasingly adopted in industrial LLM post-training [10, 32, 39]. Rollout and training run on separate GPU pools and overlap by allowing rollout to use weights with bounded staleness. This overlap reduces waiting from strict dependencies, but the throughput imbalance remains: when rollout is slower than training, training GPUs idle after each update

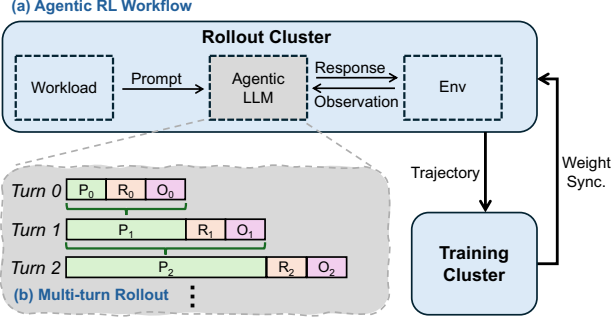


Figure 1. Agentic RL workflow and multi-turn rollout.

while waiting for the next trajectory batch. Rollout efficiency therefore remains critical.

2.2 Resource Elasticity in Agentic RL

Beyond asynchronous execution, rollout can exploit elastic GPUs. Public cloud providers offer discounted preemptible instances that add capacity when available but may be reclaimed [2, 11, 33]. Production clusters similarly share GPUs across priorities: DeepSeek-V4 uses a preemptible rollout service [5], Seed provisions workers on spot GPUs [42], and Kimi and Prime Intellect scale rollout onto idle nodes [31, 32]. Academic systems such as RLBoost [38] and ROSE [7] harvest preemptible and serving slack. In these settings, the rollout cluster must resize as resources come and go.

3 Motivation

3.1 Underutilization of Elastic Resources

To assess whether existing systems effectively exploit elastic resources, we characterize RLBoost+, our extension of RLBoost [38] that supports agentic RL and asynchronous training, as described in the Section 6.1. We run Qwen3-8B [40] on SWE-bench [16] with 8 training GPUs and scale rollout from 16 to 32 GPUs through data parallelism (DP). The trajectory batch is partitioned across rollout workers, each hosting a replica of the model weights. The total trajectory batch size and per-worker model parallelism remain unchanged. Figure 2a reveals resource underutilization, concerning additional external GPUs and the GPUs already allocated.

Limited efficiency of external scale-out. As shown in Figure 2a, provisioning additional GPUs reduces rollout time and consequently improves rollout throughput. However, the marginal benefit diminishes as the GPU allocation increases. Scaling from 16 to 24 GPUs increases the allocated resources by 50% but reduces rollout latency by only 10.3%. Further scaling to 32 GPUs yields an additional latency reduction of merely 8.9%, with both reductions normalized to the rollout time on 16 GPUs. Therefore, the performance gains fall substantially short of proportional scaling, because data parallelism partitions the trajectory batch but replicates the

model weights across workers. Each worker processes fewer trajectories while still reading the same model weights during decoding, amortizing weight-access costs over a smaller batch. To assess whether scale-out alone can eliminate the bottleneck, consider an idealized scenario with unlimited rollout GPUs. Each DP worker processes one trajectory concurrently and per-worker model parallelism remains unchanged. Using the measured mean trajectory duration as an optimistic estimate, rollout still takes 216.3 s, compared with 163.8 s for training. This indicates that continuously adding rollout resources increases resource consumption without eliminating the rollout bottleneck.

Idle capacity in the training pool. The same imbalance also leaves training resources underutilized. Figure 2a shows that rollout takes 4.5–5.7× as long as training. Although asynchronous execution overlaps both stages, training GPUs become idle after completing an update and wait for enough new trajectories to form the next batch. Under the idealized scale-out estimate above, rollout still takes approximately 1.3× as long as training, leaving an estimated 24.3% of training-GPU time idle in this workflow. These recurring idle windows provide additional capacity that can temporarily accelerate rollout before the GPUs return to training.

We refer to changes in rollout capacity obtained from outside the job’s reserved GPU pool as inter-scale elasticity, and to temporary reuse of idle training GPUs within that pool as intra-scale elasticity. Both determine the resources available to rollout and must be considered jointly.

3.2 Efficiency Degradation under PD Colocation

We further investigate why rollout remains expensive despite additional GPU resources. Existing RL frameworks, including VeRL [26], ROLL [35], and slime [46], use LLM serving engines for rollout generation. A common deployment colocates prefill and decode on the same GPUs and adopts standard serving optimizations. Chunked prefill [1] divides long prefills into smaller chunks and batches them with decode requests, reducing decode stalls and peak activation memory usage. Prefix caching [9] reuses KV states for previously processed prefixes, avoiding redundant computation across requests and interaction turns.

However, our measurements show that prefill still repeatedly interrupts decoding in agentic rollout, even with these optimizations enabled. This is because each environment interaction introduces new observations that require another prefill before generation resumes. Frequent prefill arrivals therefore continue to delay decoding and reduce generation throughput. Figure 2b illustrates this behavior for Qwen3-30B-A3B [40] on SWE-bench. Decode throughput stays near 400 tokens/s per GPU between insertions but drops on prefill arrival. In this interval, prefill processes 96.68% of tokens in 26.48% of the elapsed time, while decode produces only 3.32% of tokens but 73.52% of the time. Decoding thus dominates execution time, yet prefill repeatedly disrupts it.

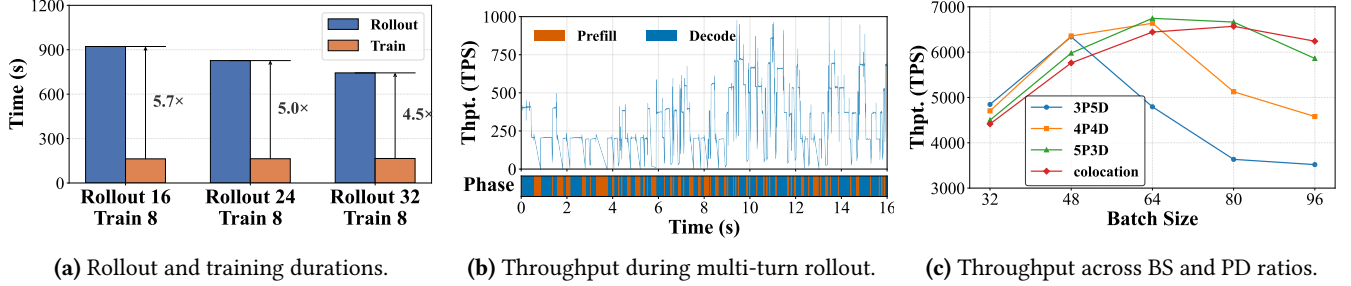


Figure 2. Computational characteristics of agentic RL workflows. (a) Rollout and training durations for Qwen3-8B as rollout resources increase from 16 to 32 GPUs, with training fixed at 8 GPUs. (b) Generation throughput of Qwen3-30B-A3B during multi-turn rollout. (c) Rollout throughput across batch sizes and PD ratios for Qwen3-30B-A3B.

Consequently, adding GPUs under the same colocated strategy does not eliminate prefill–decode interference within each worker. This interference limits the effective use of resources from both inter-scale expansion and intra-scale reuse. Exploiting these resources therefore requires addressing PD interference alongside resource allocation.

3.3 Opportunity of PD Disaggregation

PD disaggregation places prefill and decode on separate GPU sets to eliminate interference [45]: prefill workers process prompts and transfer the KV cache to decode workers, allowing both phases to execute concurrently.

To assess its benefit for rollout of agentic RL, we extend ROLL [35] with disaggregated execution and compare throughput for Qwen3-30B-A3B on SWE-bench across batch sizes, using colocation and different PD ratios (Figure 2c). All use 16 GPUs as 8 workers with tensor-parallelism [27] degree 2.

First, the preferred execution mode depends on the workload. The best disaggregated configuration outperforms colocation by 9.7% at BS32, but the advantage narrows to 1.4% at BS80 and reverses to a 6.1% colocation win at BS96. Disaggregation trades reduced interference for KV-transfer overhead and a partitioned GPU pool; when transfer overhead dominates or either pool bottlenecks, colocation can be preferable.

Second, the optimal PD ratio depends on the workload. No single ratio is best across batch sizes: the best changes from 4P4D at BS48 to 5P3D at BS64. Shifting GPUs to prefill increases its concurrency, but under a fixed budget it leaves fewer decode GPUs, potentially making decode the bottleneck. A static ratio is therefore insufficient.

These observations connect PD configuration to resource elasticity: added GPUs change per-worker load and feasible allocations, so the execution mode and PD ratio must be re-selected from the current workload and GPU budget.

3.4 Limitations of Existing Solutions

The preceding observations show that exploiting elastic resources requires adapting both rollout capacity and PD execution. The preferred execution mode and P/D ratio depend

Table 1. Comparison of rollout deployment adaptation and PD configurations in representative RL systems. Resource adaptation denotes inter-scale elasticity and intra-scale training-GPU reuse; workload adaptation denotes deployment changes in response to workload.

Category	System	Adaptation		PD Configuration	
		Resource	Workload	Mode	P/D Ratio
Fixed & Async RL	AReAL [6]	✗	✗	Colocated	–
	ROLL [35]	✗	✗	Colocated	–
	Laminar [25]	✗	✗	Colocated	–
Elastic RL	RLBoost [38]	Inter-scale	✓	Colocated	–
	ROSE [7]	Inter-scale	✓	Colocated	–
	SeamlessFlow [34]	Intra-scale	✗	Colocated	–
	BiDiRL [29]	Intra-scale	✗	Colocated	–
PD-Disaggregated RL	RollArt [8]	✗	✗	Coloc. or Disag.	Static
	NeMo RL [28]	✗	✗	Coloc. or Disag.	Static
	Slime [46]	✗	✗	Coloc. or Disag.	Static
	PEARL	Both	✓	Coloc. ↔ Disag.	Dynamic

on the workload and available resources. Table 1 summarizes how representative RL systems address these requirements. **Fixed and asynchronous RL systems.** AReAL [6], ROLL [35], and Laminar [25] improve training efficiency by overlapping rollout and policy updates. Their rollout deployments use fixed resource pools with colocated prefill and decode. Asynchronous execution alone does not enable these deployments to absorb external GPU capacity or reuse idle training GPUs, leaving both forms of resource elasticity unexploited.

Elastic RL systems. RLBoost [38] and ROSE [7] provide inter-scale elasticity through external preemptible GPUs and spare serving capacity, respectively. SeamlessFlow [34] and BiDiRL [29] provide intra-scale elasticity by reusing training resources for rollout. These systems demonstrate complementary ways to obtain additional capacity, but retain colocated PD execution. As §3.3 shows, the preferred execution mode changes with workload. Adding rollout capacity alone therefore does not resolve whether prefill and

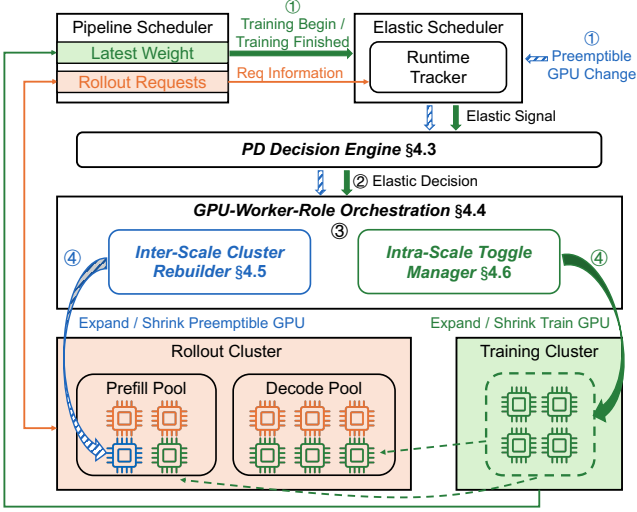


Figure 3. Architecture of PEARL.

decode should remain colocated or how resources should be divided between them.

PD-disaggregated RL systems. RollArt [8], NeMo RL [28], and Slime [46] support PD-disaggregated rollout, but use statically configured execution modes and P/D ratios. A configuration chosen at launch need not remain efficient when external GPUs arrive or are reclaimed, or when training GPUs become available for rollout and are later returned. Supporting PD disaggregation alone does not provide adaptation to the changing resource budget and workload.

Why not simply combine existing pieces? Combining elasticity with PD disaggregation requires coordinated decisions and transitions. Resource changes alter both the feasible P/D allocations and the workload per worker, so configuration selection must track the current resource state. Applying a choice requires handling in-flight requests, synchronizing weights, and changing worker roles while meeting resource-reclamation and training-resumption requirements. These transitions incur costs that can outweigh the gains from optional expansion or borrowing. PEARL therefore couples workload-aware configuration selection with cost-aware reconfiguration, jointly considering inter-scale and intra-scale resources to reduce rollout-batch makespan.

4 Design

4.1 System Overview

4.1.1 Architecture. Figure 3 shows the architecture of PEARL. The Pipeline Scheduler runs the asynchronous RL hot path: its router dispatches rollout requests to the serving workers, and it collects trajectories and synchronizes weights. The Elastic Scheduler reacts to Training Begin/Finished notifications and preemptible-GPU changes. Its Runtime Tracker maintains a workload profile from request telemetry. The

PD Decision Engine uses the profile and the current rollout GPU budget to choose an execution mode (colocation or disaggregation) and role allocation that minimizes predicted batch makespan. GPU-Worker-Role Orchestration turns that allocation into an incremental reconfiguration plan, run by two executors: the Inter-Scale Cluster Rebuilder for preemptible-GPU changes and the Intra-Scale Toggle Manager for training-GPU reuse.

The rollout cluster generates trajectories with prefill, decode, and colocated workers, while the training cluster keeps fixed workers and GPU bindings. Training GPUs can be borrowed for rollout during idle windows (§4.6). The remainder of this section formalizes the optimization, then details the decision engine, orchestration, and the two executors.

4.1.2 Workflow. All elastic events, including training notifications and preemptible-GPU changes, traverse the same four stages. The Elastic Scheduler updates the rollout budget ①. The PD Decision Engine selects the execution mode and role allocation ②. GPU-Worker-Role Orchestration produces an incremental reconfiguration plan ③. The responsible module executes it ④. The two triggers differ only in how the budget changes and in the handoff constraints.

A complete training step. During rollout, the Pipeline Scheduler dispatches requests while the Runtime Tracker profiles the workload. Training needs a complete batch, so training GPUs sit idle until the remaining trajectories finish. The Toggle Manager can borrow them during this window (§4.6). Once the batch is ready, Training Begin triggers a shrink: borrowed training GPUs are reclaimed, leaving only dedicated rollout GPUs ①, the PD Decision Engine re-selects mode and roles ②, Orchestration schedules their return first ③, and the Intra-Scale Toggle Manager drains their requests, retains partial trajectories, and releases the GPUs ④. Dedicated rollout workers keep serving.

After training updates the model and the Pipeline Scheduler syncs new weights, Training Finished triggers a grow: the budget again includes all training GPUs ①, the same decision and orchestration path produces a plan ②–③, and the Intra-Scale Toggle Manager admits workers on borrowed training GPUs only if the benefit-aware gate permits and their runtime state and weight versions are ready for serving ④. Otherwise rollout continues on dedicated workers alone.

Changes in preemptible-GPU availability. Preemptible-GPU changes use the same stages ① through ③, and only execution differs. For a shrink, the Inter-Scale Cluster Rebuilder removes the reclaimed workers from routing, drains them as in the training-triggered shrink, and returns their GPUs. If this breaks the PD topology, the decision stage already picks a feasible replacement (§4.2). For an expansion, it initializes workers in the background and admits them only after weight sync, so requests never hit a half-ready worker. Only affected workers change, and the rest keep serving.

Training-GPU borrowing and preemptible-GPU scaling differ: returning borrowed GPUs is mandatory because training cannot wait, while borrowing and expansion are optional and happen only when predicted gain exceeds handoff cost. Since both share one decision and execution path, the workload, not the GPU source, decides whether added resources become prefill, decode, or colocated workers.

4.2 Problem Formalization

Unified optimization view. We formalize the choice of execution mode and role allocation under a time-varying budget as one optimization problem. The system chooses whether to disaggregate prefill and decode or colocate them and, under disaggregation, each role’s worker count.

Objective. Because rollout is the bottleneck in our workloads, PEARL improves training throughput by reducing trajectory-batch completion time rather than per-request latency. We therefore use the batch makespan $T_{\text{roll}}(\pi)$, the completion time of the last of the B trajectories under configuration π , as the online criterion, and select the feasible configuration π^* with the minimal predicted makespan, resolving as the workload and resource budget change.

Decision variables and feasible configurations. For rollout step k , let N_k denote the number of currently available rollout workers. Inter-scale and intra-scale events update this budget, and the Runtime Tracker supplies the latest workload profile and platform parameters. The decision variables are $\pi = (N_P, N_D, N_C)$ with $N_P + N_D + N_C = N_k$, where N_P , N_D , and N_C are the numbers of active prefill, decode, and colocated workers. The number of training workers N_T is fixed, so the complete target configuration is $(N_T, N_P^*, N_D^*, N_C^*)$. A configuration is either fully colocated ($N_P = N_D = 0$) or fully disaggregated ($N_C = 0$ with $N_P, N_D \geq 1$). The feasible set $\Pi'(N_k)$ contains the assignments satisfying the capacity constraints below. If a shrink would remove all prefill or all decode workers, the system either reassigns surviving workers or falls back to colocation.

Capacity constraints. We obtain the feasible set $\Pi'(N_k)$ by enforcing two capacity constraints. The prefill workers must handle the token workload $\lambda_P = B \cdot S \cdot \Delta L$ per step without building queues that delay decode. Under disaggregation, the decode workers must also hold the entire batch’s KV cache: $B \cdot K_{\text{seq}} \leq N_D \cdot H_{\text{max}}$, where K_{seq} is the KV-cache size per trajectory and H_{max} is the HBM capacity per worker.

4.3 PD Decision Engine

The PD Decision Engine solves this problem by building closed-form makespan models for colocation and disaggregation from hardware specs, model architecture, and runtime telemetry, enumerating the feasible disaggregated configurations, and selecting the one with the minimal predicted batch makespan.

4.3.1 Bubble-Aware Makespan Models. Within a rollout step, each trajectory alternates between prefill insertions and decode, so prefill and decode requests interleave across the batch. They compete for the same GPUs under colocation but run on separate pools under disaggregation. We therefore model the two modes separately.

PD colocation. Prefill and decode execute serially on shared GPUs. We model the batch makespan as

$$T_{\text{coloc}} = S \left[\underbrace{\frac{B \cdot \Delta L}{N} \cdot t_{\text{prefill}}}_{T_{\text{prefill}}^{\text{turn}}} + r \underbrace{\left(A + b \frac{B}{N} \right)}_{T_{\text{decode}}^{\text{turn}}} + T_{\text{over}} \right] + T_{\text{env}}. \quad (1)$$

The batch has B trajectories, $N = N_k$ rollout workers, and S interaction turns on the slowest trajectory. Each turn adds ΔL prompt tokens and generates r tokens. The per-token prefill time t_{prefill} is the computation per token divided by GPU compute capacity. In the decode term, A is the fixed per-iteration cost, including weight reads and kernel dispatch, and b is the marginal cost of each additional concurrent sequence, mainly from extra KV-cache HBM reads. T_{over} covers overheads such as switching between prefill and decode, and T_{env} is the environment interaction time.

The model fixes the decode batch size at B/N . It assumes work conservation: when trajectories leave decode for environment interactions, prefill consumes the freed GPU capacity. The model therefore captures these interactions through S and T_{env} without per-turn concurrency correction.

PD disaggregation. Under disaggregation, the N_k rollout workers are split into N_P prefill workers and N_D decode workers, with $N_P + N_D = N_k$. Prefill and decode run on separate GPU pools and can execute concurrently. During environment interactions, trajectories sit in neither queue, so the effective decode batch size drops. Unlike colocation, disaggregated decode GPUs cannot reuse idle cycles for prefill, so environment interactions reduce decode throughput.

We quantify this effect by the average fraction of trajectories that are actively decoding, f , or equivalently the bubble ratio $\theta = (1 - f)/f$. The effective decode concurrency then drops from B to $B/(1 + \theta)$. The decode-path batch makespan is

$$T_{\text{decode}} = S \cdot r \cdot \left(A + b \frac{B}{N_D(1 + \theta)} \right) (1 + \theta). \quad (2)$$

The prefill-path batch makespan is

$$T_{\text{prefill}} = S \left(\frac{B \cdot \Delta L}{N_P} \cdot t_{\text{prefill}} + \frac{B \cdot \Delta L \cdot K}{\text{BW}} \right). \quad (3)$$

t_{prefill} is the same as in the colocated model, K is the KV transfer volume per token, and BW is the available bandwidth between the two pools. The second term lower-bounds the cross-pool KV-transfer time because it assumes ideal, uncontended use of the full bandwidth. The disaggregated

makespan is the longer of the two paths plus environment interaction time, $T_{\text{dis}} = \max(T_{\text{prefill}}, T_{\text{decode}}) + T_{\text{env}}$.

4.3.2 Online Decision and Switching Control. After each rollout step, the Runtime Tracker refits the makespan-model parameters ($S, r, \Delta L, \theta, A, b$, and t_{prefill}) from request telemetry. Upon each elastic signal, the PD Decision Engine enumerates feasible disaggregated configurations over $N_P \in [1, N_k - 1]$ (with $N_D = N_k - N_P$), discards those that violate the capacity constraints in §4.2, compares the best against colocation, and returns the target configuration. The orchestration layer then executes the required incremental changes (§4.4).

Applying a new configuration is not free: role rebinding, router updates, and CUDA graph recapture can take tens of seconds, so when the current and target configurations have similar predicted makespans, the transition can cost more than it saves. The PD Decision Engine therefore applies mode-dependent hysteresis: a switch is admitted only when the alternative configuration’s predicted makespan is lower by more than a threshold, with a larger threshold for colocation–disaggregation mode switches than for ratio adjustments within the same mode. Hysteresis suppresses thrashing from performance noise, but does not block forced switches needed to keep a feasible role configuration when the current topology becomes infeasible.

4.4 GPU–Worker–Role Orchestration

The PD Decision Engine outputs a target configuration that specifies how many workers each role needs, but not which physical workers should take these roles or how to reach the target from the current configuration. GPU–Worker–Role Orchestration solves this placement and transition problem: it maintains a consistent GPU–worker–role mapping across elastic events while minimizing reconfiguration overhead.

The two kinds of elastic resources have different lifecycles: preemptible GPUs physically join and leave the cluster, while training GPUs stay in place and only change which phase executes on them. The Orchestration therefore partitions GPUs into two zones. The dedicated rollout zone holds the GPUs reserved for rollout and expands or shrinks as preemptible instances join and leave. The overlap zone holds the training GPUs, which rollout can borrow during idle windows. Rollout workers placed in this zone are *overlap workers*. Training workers and their GPU bindings remain fixed: borrowing changes execution ownership, not the training topology. The GPUs available for rollout at any time are the union of the currently available GPUs in the two zones.

Elastic events in these zones pose two challenges: concurrent events can leave different system components with conflicting views of the same worker, and rebuilding each target configuration from scratch turns a local change into global overhead. The Orchestration addresses them with two

mechanisms, a unified GPU–worker–role state and minimal-change plan generation.

4.4.1 A Unified GPU–Worker–Role State. Inter-scale events, intra-scale events, and role adjustments can all arrive during the same rollout. If the router, the executors, and the Pipeline Scheduler keep separate views, they may disagree on whether a worker is serving, transitioning, or due back to training. The result can be requests routed to departing workers or training and rollout contending for the same GPU memory. The Orchestration therefore represents resource state as a single unified GPU–worker–role mapping shared by all system components. The mapping has two links: the GPU-to-worker link binds each worker to a fixed set of physical GPUs, and the worker-to-role link records each worker’s current role and activity status. This single source of truth links the PD Decision Engine’s target configuration to concrete execution and prevents different system components from interpreting it independently. Logically, the state is a table keyed by stable worker identifiers w :

$$\mathcal{M} = \{w \mapsto (G_w, R_w, a_w)\}, \quad (4)$$

where G_w is the GPU set bound to worker w , R_w is its role (prefill, decode, colocated, or training), and a_w is its membership status in the active set: *active* workers currently receive and serve rollout requests (the set $\mathcal{W}$), *inactive* workers keep their processes alive but serve no requests (e.g., an overlap worker whose GPUs are currently running training), and *transitioning* workers are being added to or removed from the active set, or reassigned, and must drain or finish initialization before they can serve.

Decoupling physical bindings from logical roles. The mapping keeps physical bindings stable and lets only logical roles change. Each worker occupies a stable slot in the table and binds to a fixed set of GPUs for its lifetime, so scaling never rennumbers surviving workers and new workers simply reuse empty slots. Fixed bindings still allow time-sharing: in the overlap zone, a training worker and an overlap rollout worker share the same G_w , and the role R_w and activity a_w determine which phase currently uses the GPUs. Rollout workers can change roles or leave and rejoin the active set without altering their bindings. The three forms of dynamics therefore reduce to two logical updates on this state: changing the active worker set (membership) or reassigning roles among surviving workers. Mapping updates are serialized so concurrent events never apply to stale state, while reconfiguration operations execute asynchronously.

4.4.2 Minimal-Change Plan Generation. Rebuilding each target configuration from scratch recreates workers that need not change. Worse, a rebuilt cluster rennumbers the surviving workers, and communication groups, request routing, and weight broadcasts all reference workers by index, so every survivor must rejoin these structures. A change that

touches a few workers then becomes a cluster-wide reinitialization. Placements with identical role counts are therefore not equivalent. The orchestration layer instead turns the target configuration into a plan that preserves existing bindings and roles wherever possible. Stable slots separate membership changes from role changes among surviving workers. For current and target active sets $\mathcal{W}$ and $\mathcal{W}'$, the workers in $\mathcal{W}' \setminus \mathcal{W}$ must join and those in $\mathcal{W} \setminus \mathcal{W}'$ must leave. For each role r , let $n_{\text{src}}(r)$ and $n_{\text{dst}}(r)$ be its counts in the surviving set $\mathcal{S} = \mathcal{W} \cap \mathcal{W}'$ and in the target configuration. At most $\min(n_{\text{src}}(r), n_{\text{dst}}(r))$ survivors can keep role r , and the excess must switch. The change count therefore decomposes into membership and role changes:

$$D_{\text{worker}} = |\mathcal{W}' \setminus \mathcal{W}| + |\mathcal{W} \setminus \mathcal{W}'|, \quad (5)$$

$$D_{\text{role}} = \sum_r \left[n_{\text{src}}(r) - \min(n_{\text{src}}(r), n_{\text{dst}}(r)) \right]. \quad (6)$$

Because the two worker sets are disjoint, no placement can trade a membership change for a role change: every join and every departure is fixed by the target configuration, and among the survivors the excess of each role must switch. Keeping each existing role up to its target count and filling the remaining deficits with added workers and reassigned survivors therefore achieves exactly the minimum change count $D_{\text{worker}} + D_{\text{role}}$.

The orchestration layer emits a concrete reconfiguration plan $(\mathcal{W}_{\text{add}}, \mathcal{W}_{\text{remove}}, \rho)$ to the execution modules, where $\mathcal{W}_{\text{add}}$ and $\mathcal{W}_{\text{remove}}$ capture membership changes and ρ maps surviving workers to new roles. Because both the Inter-Scale Cluster Rebuilder and the Intra-Scale Toggle Manager consume the same plan format, the two GPU sources share a single execution path; they differ only in the urgency of returning GPUs.

4.5 Inter-Scale Cluster Rebuilder

The Rebuilder handles changes in the dedicated rollout zone caused by preemptible-GPU arrivals and reclamations, executing the add/remove portions of the orchestration plan while rollout continues on unaffected workers.

Reconfiguration is inherently asymmetric. Shrinking is cheap and urgent: reclaimed GPUs must be returned promptly, but interrupted workers may hold partial trajectories. Expansion is expensive and deferrable: new workers pay a large cold-start cost and cannot be admitted until fully initialized with the latest weights, yet rollout cannot pause to wait.

Shrinking. The Rebuilder stops dispatching new requests to the affected workers, interrupts unfinished requests while retaining partial trajectories, and returns the GPUs synchronously. Subsequent execution resumes from these partial trajectories without interrupting rollout on other workers.

Expansion. New workers initialize in the background while existing workers continue serving. Because training may update weights during this window, the Rebuilder uses *lazy weight propagation*: it maintains the latest available weight

snapshot and synchronizes new workers only after they initialize, avoiding both stale startup checkpoints and frequent checkpoint writes. Once all added workers have finished initialization, weight synchronization, and runtime preparation, the Rebuilder applies *atomic finalization* to admit the entire group at once. This pauses admission of new requests briefly but does not block requests already executing. Finally, an *all-or-nothing admission* policy cancels the whole expansion if any worker fails to initialize, preventing a partial topology from being exposed to the router.

4.6 Intra-Scale Toggle Manager

The Manager borrows training GPUs during idle windows between training steps, time-sharing existing training GPUs with rollout rather than adding physical GPUs.

Idle compute alone does not ready a GPU for rollout because training and rollout share the same memory budget; pausing training does not free rollout memory. Repeated process destruction at every handoff would incur cold-start costs that erase the benefit within short windows.

Process Retention and Device-State Handoff. The Toggle Manager decouples process lifetimes from device-state residency. It retains training and rollout processes across handoffs and swaps only the device state needed for the active phase: training releases GPU memory when borrowing begins, and overlap workers drain before returning. Returning is mandatory because training cannot proceed until borrowed GPUs are released. Rollout on dedicated GPUs continues uninterrupted.

Benefit-aware activation. Process retention reduces repeated initialization but not activation and return costs. The Toggle Manager therefore activates borrowing only when the predicted benefit exceeds the handoff cost. It compares ded, using only dedicated rollout resources, with full, using all training GPUs as overlap workers. With R of the B trajectories not yet completed, it enables borrowing when

$$\frac{R}{B} (T_{\text{ded}} - T_{\text{full}}) > T_{\text{toggle}}, \quad (7)$$

where T_{toggle} is the incremental cost of activation and return. This cost-aware gate prevents the Toggle Manager from paying handoff overhead for borrowing windows that are too short to amortize it.

5 Implementation

PEARL extends ROLL with elastic rollout management, weight synchronization, and runtime workload tracking, using SGLang’s [44] native PD disaggregation for rollout workers. For intra-scale toggle, SGLang’s integrated torch memory saver releases resident weights, KVCache, and CUDA graphs [24, 45] when training begins while retaining the rollout process; these states are restored when GPUs become available. Under PD disaggregation, restoration also refreshes communication metadata for KV buffers. Interrupted requests preserve their

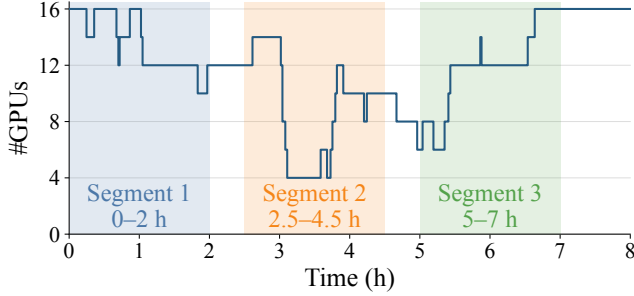


Figure 4. Dedicated rollout GPU availability and the three two-hour segments used in our experiments. The count includes 4 reserved GPUs and up to 12 preemptible GPUs; the 16 reserved training GPUs are excluded.

generated-token prefix and interaction progress in the environment, then resubmit the prefix to an available worker to resume generation.

Training workers. PEARL extends ROLL’s Megatron [27] and FSDP [43] backends with complete weight snapshots and topology-aware weight updates. Snapshots reuse weights aggregated during normal updates, avoiding additional checkpoints. The controller stores only version, metadata, and references. New rollout workers asynchronously fetch the snapshot, load it, and hand it to the local inference process.

Worker lifecycle. PEARL manages training and rollout workers as Ray [22] actors, tracking their GPU bindings, roles, and service states. External resource managers submit inter-scale changes through Ray, while the training pipeline triggers intra-scale borrowing and returns without recreating workers.

Runtime statistics. For each step, PEARL records interaction turns, newly added input tokens, generated tokens, response tokens retained in the next context, and environment-processing time. It aggregates these statistics across steps to update the PD decision engine’s workload profile, exploiting similarity between adjacent steps without assuming a stationary workload. Exponential moving average [15] for toggle-enabled and toggle-disabled steps estimate switching costs online and update the intra-scale gate.

6 Evaluation

We evaluate PEARL along five dimensions: end-to-end throughput under changing GPU availability, the performance impact of PD disaggregation and intra-scale toggle, the quality of the PD Decision Engine’s ratio selection, the sensitivity to workload and reconfiguration overhead.

6.1 Experimental Setup

Models and training configuration. We train Qwen3-8B and Qwen3-30B-A3B on software-engineering tasks from SWE-bench using GRPO with the FSDP2 backend. Unless

otherwise specified, we use a group size of 16, a rollout batch size of 64, a maximum trajectory length of 32K tokens, and a maximum policy staleness of 1 training step. All rollout workers—prefill, decode, and colocated—use tensor parallelism of degree 2 (TP2).

Cluster setup. We conduct our experiments on a cluster of 32 NVIDIA H800 GPUs, each with 80 GB of GPU memory. GPU nodes are interconnected via 800 Gbps InfiniBand. We allocate 16 reserved GPUs to training, providing sufficient memory for Qwen3-30B-A3B. In the end-to-end experiments, PEARL and RLBoost+ use 4 reserved rollout GPUs and up to 12 additional preemptible GPUs, whose availability follows the replayed trace below.

Resource traces. Following RLBoost [38], we replay three representative two-hour segments from the real GPU availability trace provided by Bamboo [33]. Figure 4 shows the selected segments: Segment 1 exhibits frequent availability changes with abundant GPU resources; Segment 2 exhibits frequent changes with limited resources; and Segment 3 provides moderate resources with fewer changes.

Baselines. We compare PEARL with three baselines in the end-to-end experiments, all using the same 16 reserved training GPUs:

- **ROLL.** This asynchronous baseline uses a fixed allocation of 4 reserved rollout GPUs and no preemptible resources.
- **ROLL-Large.** This variant of ROLL uses a larger fixed pool of reserved rollout GPUs. For Segments 1–3, the mean rollout GPU counts are 13.6, 9.0, and 11.8. Rounding each up to the nearest multiple of 2 for TP2 gives fixed allocations of 14, 10, and 12 GPUs, respectively. These allocations avoid the preemption and instance-startup overheads associated with resource changes.
- **RLBoost+.** RLBoost+ is our adaptation of RLBoost [38] for asynchronous agentic RL. It uses the same reserved and preemptible rollout resources and replays the same availability traces as PEARL.

Throughput metrics. For each step, system throughput is the sum of the token counts for rollout generation and training, divided by the step duration. We report throughput in tokens per second, showing both per-step values over time and averages for each trace segment.

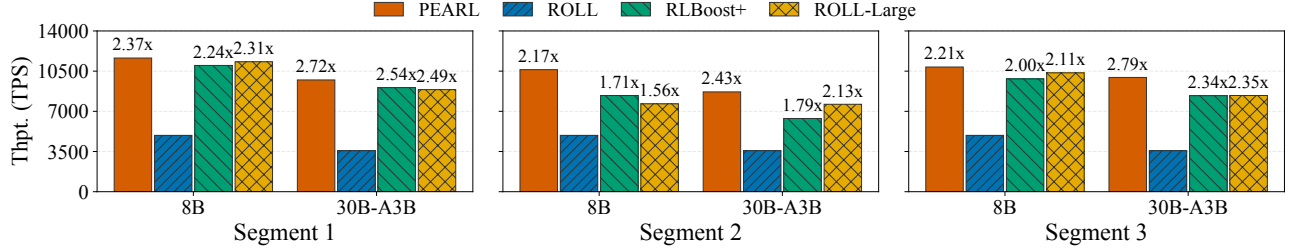


Figure 5. Average system throughput for each model and trace segment. Bar annotations are normalized to ROLL. RLBoost+ shares PEARL’s external rollout resource availability and training GPU allocation; ROLL-Large uses a fixed, larger reserved rollout pool.

6.2 End-to-End Performance

Overall throughput. PEARL improves average throughput over RLBoost+ by approximately 5.8–26.9% for Qwen3-8B and 7.4–36.3% for Qwen3-30B-A3B across the three trace segments (Figure 5). Both systems face the same external rollout resource availability and use 16 reserved training GPUs, making RLBoost+ our primary comparison for execution efficiency under changing resources. Relative to fixed-resource ROLL, PEARL achieves 2.17–2.37 $\times$ and 2.43–2.79 $\times$ the throughput for the two models, respectively. This latter comparison captures the combined benefit of additional preemptible resources and PEARL’s execution mechanisms. **Impact of resource availability.** The advantage over RLBoost+ is largest in Segment 2, where the dedicated rollout pool averages 9.0 GPUs: throughput improves by approximately 26.9% for Qwen3-8B and 36.3% for Qwen3-30B-A3B. In Segment 1, with 13.6 rollout GPUs on average, the improvements narrow to approximately 5.9% and 7.4%. Segment 3 lies between these cases, averaging 11.8 rollout GPUs and yielding gains of approximately 10.6% and 19.0%. The larger gains under constrained rollout capacity are consistent with the fixed-resource ablations in §6.3, which show greater benefit from reusing idle training GPUs when dedicated rollout capacity is limited. The trace comparisons establish this pattern across the evaluated conditions; they do not isolate the contribution of each mechanism.

Comparison with fixed provisioning. PEARL is also competitive with ROLL-Large, which uses 14, 10, and 12 reserved rollout GPUs for Segments 1–3, respectively. These fixed allocations slightly exceed each segment’s average dedicated rollout capacity and avoid preemption and resource-change startup overheads. PEARL achieves higher measured average throughput for both models in all three segments, although its Qwen3-8B result in Segment 1 is close to the fixed deployment. ROLL-Large therefore provides a stable-provisioning reference; its fixed allocation does not represent a performance upper bound or an identical resource schedule.

Behavior under resource changes. Figure 6 shows throughput evolution under the same trace segments. During the pronounced resource reduction in the middle of Segment 2,

RLBoost+’s per-step throughput drops more sharply than PEARL’s for both models. As resources return, RLBoost+’s throughput recovers and the gap narrows. This interval illustrates where PEARL sustains its advantage within the trace, complementing the segment averages. Because throughput is measured per step, these curves do not resolve instantaneous reconfiguration latency. The following ablation study examines the separate contributions of PD disaggregation, intra-scale toggle, and its benefit-aware gate.

6.3 Ablation Study

To isolate the contributions of PD disaggregation, intra-scale toggle, and the intra-scale gate, we evaluate Qwen3-8B and Qwen3-30B-A3B under fixed resource allocations. Each experiment uses 16 training GPUs and 4 to 16 dedicated rollout GPUs, with the same training parameters as the end-to-end experiments. Figure 7 reports each configuration’s throughput relative to the ROLL baseline. We then add, in order, PD disaggregation with the optimal PD ratio, ungated intra-scale toggle, and the benefit-aware intra-scale gate, with the final configuration corresponding to PEARL.

With 4 rollout GPUs, the PD Decision Engine would select PD colocation. For this ablation, however, we force the PD-disaggregated configuration so that the figure exposes the cost and benefit of PD disaggregation at this resource point. PD disaggregation reduces throughput by 20.6% for Qwen3-8B and 30.9% for Qwen3-30B-A3B. This high compute pressure also makes intra-scale toggle highly effective: adding the ungated toggle raises throughput by 93.6% and 118.3%, respectively. Adding the gate further increases the Qwen3-30B-A3B gain to +129.5%. For Qwen3-8B, the initial estimate of toggle overhead was too high, so the gate conservatively disabled toggle during several early phases that would have benefited from it; the gated configuration therefore achieves +92.3%, slightly below the +93.6% of ungated toggle. The Qwen3-30B-A3B result with 16 rollout GPUs shows the same effect.

Across 8–16 rollout GPUs, PD disaggregation improves throughput by 12.8–15.3% for Qwen3-8B and 7.8–8.9% for Qwen3-30B-A3B. Adding ungated intra-scale toggle changes

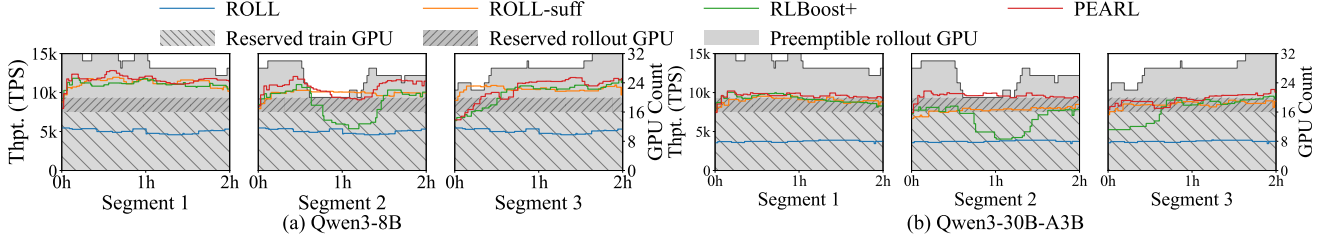


Figure 6. Per-step system throughput over three segments. The stacked background shows the shared physical GPU budget: 16 reserved training GPUs, 4 reserved rollout GPUs, and up to 12 preemptible rollout GPUs. PEARL and RLBoost+ can utilize all the GPUs, while ROLL uses the reserved allocation and ROLL-Large uses a fixed rollout pool of 14, 10, or 12 GPUs.

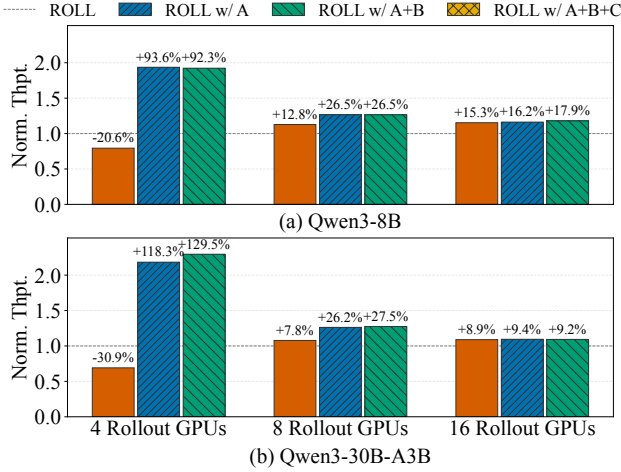


Figure 7. Ablation study with 16 training GPUs and fixed allocations of 4–16 rollout GPUs. Throughput is normalized to ROLL. A: optimal PD-disaggregation ratio; B: ungated intra-scale toggle; C: benefit-aware intra-scale gate. PEARL combines all three components.

these gains to 16.2–26.5% and 9.4–26.2%, respectively. With the intra-scale gate, the gains become 17.9–26.5% for Qwen3-8B and 9.2–27.5% for Qwen3-30B-A3B.

These results indicate that the mechanisms address different resource conditions. When dedicated rollout capacity is limited, intra-scale toggle borrows idle training GPUs and offsets the high compute pressure of rollout. As dedicated rollout capacity grows, the benefit of toggle decreases, while PD disaggregation remains useful because it removes prefill-decode interference when computation is less constrained. The gate may slightly reduce the measured gain in some situations, but it prevents toggle from being used when its estimated cost exceeds its benefit; in all tested configurations, the gated system remains faster than the baseline. Together, PD disaggregation, intra-scale toggle, and the gate provide positive throughput gains across both models and all tested resource allocations.

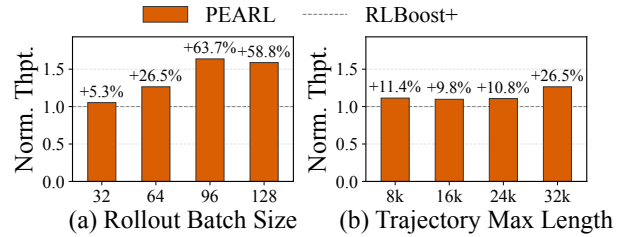


Figure 8. Workload sensitivity. (a) Rollout batch size varies with the maximum trajectory length fixed at 32K tokens. (b) Maximum trajectory length varies with the rollout batch size fixed at 64.

6.4 Workload Sensitivity

We evaluate how rollout batch size and maximum trajectory length affect the throughput advantage of PEARL over RLBoost+. Both experiments use Qwen3-8B with 16 training GPUs and 8 dedicated rollout GPUs, varying one workload parameter at a time. Figure 8 reports the results.

Rollout batch size. We fix the maximum trajectory length at 32K tokens and vary the per-step rollout batch size from 32 to 128. RLBoost+ reaches its highest measured throughput at batch size 64, whereas PEARL peaks at 96; throughput declines beyond these respective peaks. PEARL improves throughput over RLBoost+ by 5.3–63.7% across the tested batch sizes. This pattern is consistent with the ablation results: intra-scale toggle provides more benefit when the dedicated rollout pool faces higher compute pressure, but less benefit when the workload offers limited opportunity to use additional capacity.

Maximum trajectory length. We fix the per-step rollout batch size at 64 and vary the maximum trajectory length from 8K to 32K tokens. Both systems’ throughput decreases as the length limit increases, but PEARL remains faster at every tested limit, with gains of 9.8–26.5% over RLBoost+.

Together, the experiments show that PEARL improves throughput across the tested Qwen3-8B workloads under a fixed resource budget, while the magnitude of the gain depends on the batch size and trajectory-length limit.

Table 2. PD-ratio selection for Qwen3-8B.

Dedicated GPUs	Overlap	Selected	Empirical best
10	No	2P3D	2P3D
10	Yes	4P9D	5P8D
12	No	3P3D	3P3D
12	Yes	4P10D	4P10D
16	No	3P5D	3P5D
16	Yes	4P12D	4P12D

Table 3. Mean reconfiguration latency in seconds.

Operation	Qwen3-8B	Qwen3-30B-A3B
Inter-scale preparation	56.29	67.06
Inter-scale finalization	11.05	17.89
Intra-scale activation	6.26	6.09
Intra-scale deactivation	3.18	4.38

6.5 Adaptation Quality and Cost

We evaluate the quality of the PD Decision Engine’s ratio selection and the execution costs of inter-scale and intra-scale reconfiguration.

6.5.1 PD Ratio Selection

We evaluate ratio selection using constant-resource intervals from the Qwen3-8B end-to-end runs with 10, 12, and 16 dedicated rollout GPUs, both with and without intra-scale borrowing. For each interval, we reproduce the resource allocation and replay the recorded generation counts and lengths to benchmark every feasible PD ratio. The ratio with the highest measured average throughput serves as the empirical best.

Table 2 shows that the selected ratios match the empirical best at all three dedicated-only resource points and at two of the three points with overlap enabled. The only mismatch occurs with 10 dedicated GPUs and overlap enabled, where the selected 4P9D and the empirical best 5P8D differ by one worker’s role. In this case, throughput measured in the system run is approximately 0.4% below the empirical optimum, although the comparison between separate runs may also reflect measurement variation.

6.5.2 Reconfiguration Cost

Table 3 reports mean operation latencies for inter-scale expansion and intra-scale handoffs. To interpret these costs, we distinguish their effects on the availability of additional rollout capacity, the admission of new requests, and the return of borrowed GPUs to training.

Inter-scale expansion. Inter-scale expansion first initializes new workers in a background preparation stage, which takes 56–67 s across the two models. Because existing workers continue rollout throughout preparation, this cost delays the availability of additional capacity without pausing the active rollout pool. The subsequent finalization stage takes

11–18 s to synchronize weights and update routing and communication groups. During finalization, admission of new requests is paused while requests already in progress continue executing.

Intra-scale activation and deactivation. Intra-scale reconfiguration avoids repeated worker initialization by retaining the training and rollout processes across handoffs and restoring or releasing their device state as needed. Activation takes approximately 6 s for both models and must complete before borrowed GPUs can contribute to rollout. Deactivation takes 3.2–4.4 s and must finish before these GPUs can resume training; dedicated rollout workers continue serving throughout both operations. Because these costs recur with each borrowing cycle, the benefit-aware gate enables borrowing only when the predicted rollout-time saving exceeds the combined activation and return costs (§4.6). The fixed-resource ablations (§6.3) show that borrowing yields its largest throughput gains under limited dedicated rollout capacity, with smaller gains as dedicated capacity grows.

Scale-in recovery. Scale-in preserves each interrupted request’s generated prefix and interaction progress so that rollout can resume without restarting the trajectory (§5). For requests interrupted on decode workers, prefix-cache reuse limits additional prefill work to the newly generated tokens; interruptions on prefill workers require an additional full prefill pass. In our measurements, this additional full-prefill pass takes less than 1% of the time required to generate a complete trajectory, even at the maximum trajectory length. This measurement quantifies the extra prefill work needed to recover an interrupted request and does not include the other operations involved in scale-in.

7 Related Work

Agentic RL training systems. Many systems accelerate LLM RL post-training through asynchronous execution and rollout optimizations [6, 13, 25]. Agentic RL frameworks further support multi-turn interactions with external environments [8, 35]. RollArt, NeMo RL, and Slime also support PD-disaggregated rollout [8, 28, 46]. Their execution optimizations are complementary to deployment planning, which determines how the available GPUs should be used for rollout. We implement PEARL on ROLL and extend its rollout deployment to changing resource availability. PEARL selects colocated or disaggregated execution and the P/D ratio according to the workload and available GPUs, while keeping the training allocation fixed.

RL with resource adaptation. RL systems exploit two forms of resource elasticity. RLBoost [38] harvests external preemptible GPUs, while ROSE [7] shares spare serving capacity subject to serving SLOs; both provide inter-scale elasticity. RLBoost temporarily reuse training GPUs during a seeding time window before training begins. However, this mechanism does not support asynchronous execution.

More importantly, in agentic RL workloads, rollout latency is substantially higher than training latency. Consequently, even with a seeding time window, the training GPUs must still wait for rollouts after training starts, resulting in substantial idle bubbles. SeamlessFlow [34] and BiDiRL [29] exploit intra-scale elasticity by reusing training resources for rollout. BiDiRL accounts for predicted execution gains and switching costs when borrowing resources. These approaches provide different sources of rollout capacity. PEARL considers both forms of elasticity together with PD execution planning, adapting the worker set and its role allocation while accounting for the cost of transitions.

LLM serving techniques for RL rollouts. Splitwise [23] supports separated and mixed PD execution, while DOPD [18] adjusts instance counts and the P/D ratio in response to serving load. For multi-turn inference, AMPD [14] combines online prefill routing with offline resource planning, whereas PPD [17] routes append-prefill between prefill and decode instances based on latency trade-offs. These systems optimize serving latency, throughput, or goodput. PEARL instead selects the execution mode and P/D ratio to reduce rollout-batch makespan as external resources scale and training GPUs are borrowed or returned.

8 Conclusion

We present PEARL, an asynchronous agentic RL system for dynamic GPU resources that jointly adapts rollout resources, idle training-GPU reuse, and PD execution roles. It minimizes batch makespan and applies decisions through a unified orchestration layer: the Inter-Scale Cluster Rebuilder handles preemptible-GPU expansion with lazy weight propagation and atomic admission, while the Intra-Scale Toggle Manager borrows idle training GPUs via process retention and device-state handoff. On SWE-bench under preemptible-GPU traces, PEARL achieves 2.17–2.79 $\times$ the end-to-end throughput of fixed-resource ROLL and improves throughput over RLBoost+ by up to 26.9% and 36.3% for Qwen3-8B and Qwen3-30B-A3B, respectively.

References

- [1] Amey Agrawal, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav S Gulavani, and Ramachandran Ramjee. 2023. Sarathi: Efficient llm inference by piggybacking decodes with chunked prefills. *arXiv preprint arXiv:2308.16369* (2023).
- [2] Amazon Web Services. [n. d.]. Amazon EC2 Spot Instances. <https://aws.amazon.com/ec2/spot/>. Accessed: 2026-09-23.
- [3] Gheorghe Comanici, Eric Bieber, Mike Schaekermann, Ice Pasupat, Naveen Sachdeva, Inderjit Dhillon, Marcel Blistein, Ori Ram, Dan Zhang, Evan Rosen, et al. 2025. Gemini 2.5: Pushing the frontier with advanced reasoning, multimodality, long context, and next generation agentic capabilities. *arXiv preprint arXiv:2507.06261* (2025).
- [4] Jeffrey Dean, Greg Corrado, Rajat Monga, Kai Chen, Matthieu Devin, Mark Mao, Marc'aurelio Ranzato, Andrew Senior, Paul Tucker, Ke Yang, et al. 2012. Large scale distributed deep networks. *Advances in neural information processing systems* 25 (2012).
- [5] DeepSeek-AI. 2026. DeepSeek-V4: Towards Highly Efficient Million-Token Context Intelligence. (2026). [arXiv:2606.19348 \[cs.CL\]](https://arxiv.org/abs/2606.19348)
- [6] Wei Fu, Jiaxuan Gao, Xujie Shen, Chen Zhu, Zhiyu Mei, Chuyi He, Shusheng Xu, Guo Wei, Jun Mei, Jiashu Wang, Tongkai Yang, Binhang Yuan, and Yi Wu. 2025. AReaL: A Large-Scale Asynchronous Reinforcement Learning System for Language Reasoning. *CoRR* (2025).
- [7] Wei Gao, Yuheng Zhao, Dilxat Muhtar, Dakai An, Xuchun Shang, Tianyuan Wu, Lunxi Cao, Shaopan Xiong, Weixun Wang, Ju Huang, Teng Ma, Siran Yang, Jiamang Wang, Lin Qu, Bo Zheng, and Wei Wang. 2026. ROSE: Rollout On Serving GPUs via Cooperative Elasticity for Agentic RL. [arXiv:2605.06534 \[cs.DC\]](https://arxiv.org/abs/2605.06534) <https://arxiv.org/abs/2605.06534>
- [8] Wei Gao, Yuheng Zhao, Tianyuan Wu, Shaopan Xiong, Weixun Wang, Dakai An, Lunxi Cao, Dilxat Muhtar, Zichen Liu, Haizhou Zhao, Ju Huang, Siran Yang, Yongbin Li, Wenbo Su, Jiamang Wang, Lin Qu, Bo Zheng, and Wei Wang. 2026. RollArt: Disaggregated Multi-Task Agentic RL Training at Scale. In *20th USENIX Symposium on Operating Systems Design and Implementation (OSDI 26)*.
- [9] In Gim, Guojun Chen, Seung-seob Lee, Nikhil Sarda, Anurag Khandelwal, and Lin Zhong. 2024. Prompt cache: Modular attention reuse for low-latency inference. *Proceedings of Machine Learning and Systems* 6 (2024), 325–338.
- [10] GLM-5 Team. 2026. GLM-5: From Vibe Coding to Agentic Engineering. [arXiv:2602.15763 \[cs.LG\]](https://arxiv.org/abs/2602.15763)
- [11] Google Cloud. [n. d.]. About GPU Instances. <https://cloud.google.com/compute/docs/gpus/about-gpus>. Section: "GPUs on Spot VMs". Accessed: 2026-09-23.
- [12] Bingguang Hao, Maolin Wang, Zengzhuang Xu, Yicheng Chen, Cunyin Peng, Jinjie Gu, and Chenyi Zhuang. 2025. Exploring superior function calls via reinforcement learning. *arXiv e-prints* (2025), [arXiv–2508](https://arxiv.org/abs/2508.12345).
- [13] Jingkai He, Tianjian Li, Erhu Feng, Dong Du, Qian Liu, Tao Liu, Yubin Xia, and Haibo Chen. 2026. History Doesn't Repeat Itself but Rollouts Rhyme: Accelerating Reinforcement Learning with RhymeRL. In *Proceedings of the 31st ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*.
- [14] Wenhao He, Youhe Jiang, Penghao Zhao, Quanqing Xu, Eiko Yoneki, Bin Cui, and Fangcheng Fu. 2026. Efficient Multi-round LLM Inference over Disaggregated Serving. [arXiv:2602.14516](https://arxiv.org/abs/2602.14516)
- [15] J Stuart Hunter. 1986. The exponentially weighted moving average. *Journal of quality technology* 18, 4 (1986), 203–210.
- [16] Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2024. Swe-bench: Can language models resolve real-world github issues?. In *International Conference on Learning Representations*, Vol. 2024. 54107–54157.
- [17] Zongze Li, Jingyu Liu, Zach Xu, Yineng Zhang, Tahseen Rabbani, and Ce Zhang. 2026. Not All Prefills Are Equal: PPD Disaggregation for Multi-turn LLM Serving. [arXiv:2603.13358](https://arxiv.org/abs/2603.13358)
- [18] Junhan Liao, Minxian Xu, Wanyi Zheng, Yan Wang, Kejiang Ye, Rajkumar Buyya, and Chengzhong Xu. 2025. DOPD: A Dynamic PD-Disaggregation Architecture for Maximizing Goodput in LLM Inference Serving. [arXiv:2511.20982](https://arxiv.org/abs/2511.20982)
- [19] Yuhang Liu, Pengxiang Li, Congkai Xie, Xavier Hu, Xiaotian Han, Shengyu Zhang, Hongxia Yang, and Fei Wu. 2025. Infigui-r1: Advancing multimodal gui agents from reactive actors to deliberative reasoners. *arXiv preprint arXiv:2504.14239* (2025).
- [20] Zhengxi Lu, Yuxiang Chai, Yaxuan Guo, Xi Yin, Liang Liu, Hao Wang, Han Xiao, Shuai Ren, Pengxiang Zhao, Guangyi Liu, et al. 2026. Ui-r1: Enhancing efficient action prediction of gui agents by reinforcement learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 40. 17608–17616.
- [21] Run Luo, Lu Wang, Wanwei He, Longze Chen, Jiaming Li, and Xiaobo Xia. 2025. Gui-r1: A generalist r1-style vision-language action model for gui agents. *arXiv preprint arXiv:2504.10458* (2025).
- [22] Philipp Moritz, Robert Nishihara, Stephanie Wang, Alexey Tumanov, Richard Liaw, Eric Liang, Melih Elibol, Zongheng Yang, William Paul, Michael I Jordan, et al. 2018. Ray: A distributed framework for emerging {AI} applications. In *13th USENIX symposium on operating systems design and implementation (OSDI 18)*. 561–577.
- [23] Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Íñigo Goiri, Saeed Maleki, and Ricardo Bianchini. 2024. Splitwise: Efficient Generative LLM Inference Using Phase Splitting. In *51st ACM/IEEE Annual International Symposium on Computer Architecture*.
- [24] Ruoyu Qin, Zheming Li, Weiran He, Jialei Cui, Heyi Tang, Feng Ren, Teng Ma, Shangming Cai, Yineng Zhang, Mingxing Zhang, et al. 2026. Mooncake: A kvcache-centric disaggregated architecture for llm serving. *ACM Transactions on Storage* 22, 4 (2026), 1–38.
- [25] Guangming Sheng, Yuxuan Tong, Borui Wan, Wang Zhang, Chaobo Jia, Xibin Wu, Yuqi Wu, Xiang Li, Chi Zhang, Yanghua Peng, Haibin Lin, Xin Liu, and Chuan Wu. 2026. Laminar: A Scalable Asynchronous RL Post-Training Framework. In *Proceedings of the 21st European Conference on Computer Systems*.
- [26] Guangming Sheng, Chi Zhang, Zilingfeng Ye, Xibin Wu, Wang Zhang, Ru Zhang, Yanghua Peng, Haibin Lin, and Chuan Wu. 2025. Hybridflow: A flexible and efficient rlhf framework. In *Proceedings of the Twentieth European Conference on Computer Systems*. 1279–1297.
- [27] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. 2019. Megatron-lm: Training multi-billion parameter language models using model parallelism. *arXiv preprint arXiv:1909.08053* (2019).
- [28] shuyixiong. 2026. NeMo RL: TRT-LLM Prefill/Decode Disaggregation for GRPO Rollouts. GitHub pull request #4095, NVIDIA-NeMo/RL. Development implementation; draft pull request not merged into main. Accessed September 25, 2026. <https://github.com/NVIDIA-NeMo/RL/pull/4095>
- [29] Zhiqiang Tan, Maoxin Wang, Sijie Wang, Yiming Yin, Qiang Wang, Xiaowen Chu, and Shaohuai Shi. 2026. Bidirectional Resource Scheduling for Disaggregated and Asynchronous RL Post-Training. [arXiv:2607.09207](https://arxiv.org/abs/2607.09207)
- [30] Kimi Team, Yifan Bai, Yiping Bao, Y Charles, Cheng Chen, Guanduo Chen, Haiting Chen, Huarong Chen, Jiahao Chen, Ningxin Chen, et al. 2025. Kimi k2: Open agentic intelligence. *arXiv preprint arXiv:2507.20534* (2025).
- [31] Kimi Team, Angang Du, Bofei Gao, Bowei Xing, Changjiu Jiang, Cheng Chen, Cheng Li, Chenjun Xiao, Chenzhuang Du, Chonghua Liao, et al. 2025. Kimi k1. 5: Scaling reinforcement learning with llms. *arXiv preprint arXiv:2501.12599* (2025).
- [32] Prime Intellect Team, Sami Jaghouar, Justus Mattern, Jack Min Ong, Jannik Straube, Manveer Basra, Aaron Pazdera, Kushal Thaman, Matthew Di Ferrante, Felix Gabriel, et al. 2025. Intellect-2: A reasoning model trained through globally decentralized reinforcement learning. *arXiv preprint arXiv:2505.07291* (2025).

- [33] John Thorpe, Pengzhan Zhao, Jonathan Eyolfson, Yifan Qiao, Zhihao Jia, Minjia Zhang, Ravi Netravali, and Guoqing Harry Xu. 2023. Bamboo: Making preemptible instances resilient for affordable training of large {DNNs}. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*. 497–513.
- [34] Jinghui Wang, Shaojie Wang, Yinghan Cui, Xuxing Chen, Chao Wang, Xiaojiang Zhang, Minglei Zhang, Jiarong Zhang, Wenhao Zhuang, Yuchen Cao, Wankang Bao, Haimo Li, Zheng Lin, Huiming Wang, Haoyang Huang, Zongxian Feng, Zizheng Zhan, Ken Deng, Wen Xi-ang, Huaixi Tang, Kun Wu, Mengtong Li, Mengfei Xie, Junyi Peng, Haotian Zhang, Bin Chen, and Bing Yu. 2025. SeamlessFlow: A Trainer Agent Isolation RL Framework Achieving Bubble-Free Pipelines via Tag Scheduling. *arXiv:2508.11553*
- [35] Weixun Wang, Shaopan Xiong, Gengru Chen, Wei Gao, Sheng Guo, Yancheng He, Ju Huang, Jiaheng Liu, Zhendong Li, Xiaoyang Li, et al. 2025. Reinforcement learning optimization for large-scale learning: An efficient and user-friendly scaling library. *arXiv preprint arXiv:2506.06122* (2025).
- [36] Zihan Wang, Kangrui Wang, Qineng Wang, Pingyue Zhang, Linjie Li, Zhengyuan Yang, Xing Jin, Kefan Yu, Minh Nhat Nguyen, Licheng Liu, et al. 2025. Ragen: Understanding self-evolution in llm agents via multi-turn reinforcement learning. *arXiv preprint arXiv:2504.20073* (2025).
- [37] Junde Wu, Jiayuan Zhu, Yuyuan Liu, Min Xu, and Yueming Jin. 2025. Agentic reasoning: A streamlined framework for enhancing llm reasoning with agentic tools. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 28489–28503.
- [38] Yongji Wu, Xueshen Liu, Haizhong Zheng, Juncheng Gu, Beidi Chen, Z. Morley Mao, Arvind Krishnamurthy, and Ion Stoica. 2026. RLBoost: Harvesting Preemptible Cloud Resources for Cost-Efficient Reinforcement Learning on LLMs. In *23rd USENIX Symposium on Networked Systems Design and Implementation*.
- [39] Xiaomi MiMo Team. 2026. MiMo-V2.6. Online. <https://mimo.xiaomi.com/mimo-v2-6>
- [40] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. 2025. Qwen3 technical report. *arXiv preprint arXiv:2505.09388* (2025).
- [41] John Yang, Kilian Lieret, Carlos Jimenez, Alexander Wettig, Kabir Khandpur, Yanzhe Zhang, Binyuan Hui, Ofir Press, Ludwig Schmidt, and Diyi Yang. 2026. Swe-smith: Scaling data for software engineering agents. *Advances in Neural Information Processing Systems* 38 (2026).
- [42] Chenhao Ye, Huaizheng Zhang, Mingcong Han, Baoquan Zhong, Xiang Li, Qixiang Chen, Xinyi Zhang, Weidong Zhang, Kaihua Jiang, Wang Zhang, et al. 2026. TensorHub: Scalable and Elastic Weight Transfer for LLM RL Training. *arXiv preprint arXiv:2604.09107* (2026).
- [43] Yanli Zhao, Andrew Gu, Rohan Varma, Liang Luo, Chien-Chin Huang, Min Xu, Less Wright, Hamid Shojanazeri, Myle Ott, Sam Shleifer, et al. 2023. Pytorch fsdp: experiences on scaling fully sharded data parallel. *arXiv preprint arXiv:2304.11277* (2023).
- [44] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody H Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E Gonzalez, et al. 2024. Sglang: Efficient execution of structured language model programs. *Advances in neural information processing systems* 37 (2024), 62557–62583.
- [45] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. 2024. {DistServe}: Disaggregating prefill and decoding for goodput-optimized large language model serving. In *18th USENIX symposium on operating systems design and implementation (OSDI 24)*. 193–210.
- [46] Zilin Zhu, Chengxing Xie, Xin Lv, and slime Contributors. 2025. slime: An LLM post-training framework for RL Scaling. <https://github.com/THUDM/slime>. GitHub repository. Corresponding author: Xin Lv.